

HTML and Bootstrap Models

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 3

Who this book is for . . . 3

A book that stands on its own . . . 3

What you need . . . 4

Conventions . . . 4

I: The Why . . . 5

1. Pages are models . . . 6

1.1. What a model of a page gives . . . 6

1.2. No template language, no Node.js . . . 6

2. Meaning, not markup . . . 7

2.1. Expanded when written . . . 7

2.2. One model, two renderings . . . 7

2.3. Static and interactive . . . 8

2.4. Themes . . . 8

II: The How . . . 9

3. Getting started . . . 10

3.1. Two modules . . . 10

3.2. Adding the models . . . 10

3.3. Names that clash . . . 11

3.4. What the build does . . . 12

4. The HTML model . . . 13

4.1. Nodes and elements . . . 13

4.2. The document . . . 13

4.3. Attributes and escaping . . . 15

4.4. Import maps . . . 16

4.5. The generator and the structure . . . 16

4.6. Library elements . . . 17

5. Documents, themes and layout . . . 18

5.1. The vocabulary . . . 18

5.2. A document with a theme . . . 18

5.3. The grid . . . 19

6. Components . . . 21

6.1. Content . . . 21

6.2. Forms . . . 23
6.3. Navigation . . . 24
6.4. Overlays . . . 26
6.5. The demo site . . . 27
7. Navigation trees . . . 28
7.1. Large models . . . 28
7.2. The demo . . . 29
III: Looking Ahead . . . 30
8. Looking ahead . . . 31
8.1. Documents . . . 31
8.2. Documentation sites for models . . . 31
8.3. Pages that work offline . . . 31
8.4. The polymorphic UI . . . 31
Appendix A: Reference . . . 32
A.1. Packages . . . 32
A.2. Bootstrap classes . . . 32
A.3. Java classes . . . 33
Appendix B: Building this book . . . 34
Appendix C: Glossary . . . 35

Colophon

HTML and Bootstrap Models: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes two models of the Nasdanika SDK Runtime 2026.11.0: the HTML model, Maven coordinates `org.nasdanika.sdk.runtime.models.html:model:2026.11.0`, namespace URI `runtime.sdk.nasdanika.org/models/html`, and the Bootstrap model, `org.nasdanika.sdk.runtime.models.html:bootstrap:2026.11.0`, namespace URI `runtime.sdk.nasdanika.org/models/html/bootstrap` and the URIs of its `forms` and `components` packages under it. Every listing in it is included from the tests of the two modules, which are compiled and run when the book is built, and every block of generated HTML is a file that a test compares with what the generator writes.

Copyright © 2026 Pavel Vlasov.

The text of this book is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt it for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *HTML and Bootstrap Models: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

Bootstrap is released by its authors under the MIT License; Bootswatch is by Thomas Park, under the MIT License. HTML is a specification of the WHATWG. Eclipse and EMF are trademarks of the Eclipse Foundation. Java is a trademark of Oracle and its affiliates. React is a trademark of Meta Platforms. Other names may be trademarks of their owners. Their use here identifies the technologies the models relate to and implies no endorsement.

The source of this book and the Bootstrap model's tests are at github.com/Nasdanika-SDK/runtime, in `models/html/bootstrap`; the HTML model is in `models/html/model`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books, which is itself Bootswatch's Zephyr theme.

Preface

This book is about web pages as models: HTML as an EMF model, with a generator that writes it, and Bootstrap 5, with the Bootswatch themes, as a model of what its components mean, expanded to HTML when a page is written. Its point is that a page generated from a model should itself be a model before it is text: checked, traced to its sources, and transformed, and only then written.

The first part of the book is the why: what goes wrong when pages are generated as strings, and why Bootstrap is modeled by the meaning of its components and not by their markup. The second part is the how, for Java developers: the HTML model and its generator, Bootstrap documents and themes, the grid, the components for content, forms, navigation and overlays, and navigation trees for the side panels of sites. The third says what comes next.

Who this book is for

Java developers who generate documentation sites, reports and pages from models or data, and want them correct, themed and navigable without a template language or a Node.js build.

Writers of tools who need the pages they produce to be checked and traced like the rest of their output.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. It covers two modules, the HTML model and the Bootstrap model, because the second is built on the first and they are used together. *TypeScript and React Models* is close to it: the same Bootstrap model renders in React components. You do not need that book, or any other book of the series, to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The HTML model's Java module is `org.nasdanika.sdk.runtime.models.html`, with the Maven coordinates `org.nasdanika.sdk.runtime.models.html:model:2026.11.0`. The Bootstrap model's module is `org.nasdanika.sdk.runtime.models.html.bootstrap`, with the coordinates `org.nasdanika.sdk.runtime.models.html:bootstrap:2026.11.0`.
- Some familiarity with HTML and Bootstrap. Some familiarity with EMF helps: an EMF model is a set of Java interfaces generated from a metamodel, with containment, references, reflection and persistence for free.

Conventions

- **Every listing is included from a test** of the two modules and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **Generated HTML is checked.** Each block of HTML a Bootstrap listing produces is a file in the module's tests, compared with what the generator writes.
- **Status labels** mark what this edition does not have yet: **PLANNED** for designed but not built, **SKETCH** for partly built.
- **Voice.** I write in the first person.

> Editor's notes record inconsistencies between the models and their documentation found while writing. They are shown in builds for the author and left out of published editions.



The HTML and Bootstrap models had no readmes; this book is their first documentation beyond the comments in their Xcore and the chapter "From models to the web" of the Nasdanika SDK Runtime book. The listings of `BootstrapGeneratorTests` were reformatted for the book, from lambdas on one line to blocks, with the same output.

Part I: The Why

Why pages should be models before they are text, and why Bootstrap should be modeled by what its components mean. No code is needed to read this part.

Chapter 1. Pages are models

A model that the people who own it cannot read is a private notation. The web is where they read, so pages are a generation target: documentation of models, reports, sites, galleries.

Generating HTML by concatenating strings is easy to start and hard to keep correct: escaping, nesting, the order of attributes, the one closing tag that went missing. Templates are better, and still text: a template does not know that it is producing a table, and nothing can ask it.

The runtime's generators build a model of the page and let a generator write the text.

1.1. What a model of a page gives

Correctness by construction. Text is escaped when it is written, attributes are quoted, void elements have no end tag, and the end tags are where the model's containment says. A `<script>` with `</script>` in its content does not end early.

Checks before writing. A page model can be checked before it is written, by a test or by an agent: does every image have alternative text, does every link resolve, does every page of a site have the navigation panel.

Provenance. Every node of the HTML model is `Marked`: it can carry markers that point to the elements of the source model it was generated from, so a cell of a generated table can say which feature of which element it shows.

Transformation. A page model can be transformed after it is built: a navigation panel added to every page of a site after the pages were generated, a theme changed, a section moved.

Output that is the same everywhere. The generator writes the same characters on every machine, so generated pages can be compared in a review and committed.

1.2. No template language, no Node.js

Pages are built in Java, with Java's types, refactoring and tests, and written by a generator that has no dependencies beyond EMF. Nothing runs on Node.js, and nothing is downloaded at build time. Pages that use Bootstrap load its stylesheet and script from a CDN, jsDelivr, by default.

Chapter 2. Meaning, not markup

Bootstrap is markup with conventions: a modal is a `div` with the class `modal`, containing a `div` with `modal-dialog`, containing a `div` with `modal-content`, with a header, a title whose id the modal's `aria-labelledby` names, a close button with `data-bs-dismiss`, and a button somewhere else whose `data-bs-target` names the modal's id. Getting all of that right by hand, every time, is what makes Bootstrap pages fragile.

The Bootstrap model describes what a component means, not its markup. A modal has a header with a title and a close button, a body, a size, and whether it is centered and scrollable. A toggle opens it, by reference.

2.1. Expanded when written

How meaning becomes elements, classes, ARIA and data attributes, and the ids that Bootstrap's JavaScript connects, is the business of `BootstrapStructure`. It is the one place that knows Bootstrap's markup, so the pages it writes are consistent, and a change to how a component is written is a change in one place.

Components that are one element are described by annotations on their classes in the model: the tag, the classes every element of the class has, and patterns that turn features into classes, `btn-{}` for the variant of a button. Components that are more than one element, modals, accordions, dropdowns, form checks, are expanded to plain HTML by `BootstrapStructure`. Components that refer to each other by id get ids generated from their position in the model when they have none of their own.

2.2. One model, two renderings

`BootstrapStructure` is used by the HTML generator for static pages, and by the React generator of the runtime's React model for components: a Bootstrap element is an intrinsic of JSX there, rendered by the same structure. One model of a page renders both ways, as a static page and inside an interactive application, and means the same markup in both.

2.3. Static and interactive

Bootstrap appears twice in the runtime, for two ways of delivering a page. The Bootstrap model of this book is for pages written as HTML, static or served, where Bootstrap's own JavaScript does the little that is interactive: dropdowns, modals, tooltips. The React Bootstrap model, in the React modules, describes the components of the React Bootstrap library for applications. Both model the meaning of a component, a modal with a title and a size, not its markup.

2.4. Themes

Bootswatch themes are a value of a document: one enumeration literal, from `bootstrap` for Bootstrap's own look to `zephyr`, the look of this book. A page model does not change with its theme, so the same site can be generated in all of them, as the module's demo does.

Part II: The How

The two models in Java: adding them to a project, HTML and its generator, Bootstrap documents, themes and layout, components, and navigation trees for sites.

Chapter 3. Getting started

3.1. Two modules

Module	What it has
`org.nasdanika.sdk.runtime.models.html`	The HTML model, `Generator`, which writes HTML text, and `HtmlStructure`, which maps the model to elements and attributes
`org.nasdanika.sdk.runtime.models.html.bootstrap`	The Bootstrap model in five packages, `BootstrapStructure`, and `NavigationTree`

3.2. Adding the models

The runtime's models have a bill of materials, `org.nasdanika.sdk.runtime.models:bom`, which manages their versions. Import it in `dependencyManagement`, and depend on the Bootstrap model, which brings the HTML model:

```
<dependency>
    <groupId>org.nasdanika.sdk.runtime.models.html</groupId>
    <artifactId>bootstrap</artifactId>
</dependency>
```

The models are named Java modules:

Listing 1. The module declaration of the HTML model

```
module org.nasdanika.sdk.runtime.models.html {

    exports org.nasdanika.sdk.runtime.models.html;
    exports org.nasdanika.sdk.runtime.models.html.impl;
    exports org.nasdanika.sdk.runtime.models.html.util;
    exports org.nasdanika.sdk.runtime.models.html.generator;

    requires transitive org.nasdanika.sdk.runtime.models.core;

    provides EPackage with HtmlEPackageProvider;

}
```

The HTML model requires the runtime's core model transitively, for markers and the map entries

of attributes and styles.

Listing 2. The module declaration of the Bootstrap model

```
module org.nasdanika.sdk.runtime.models.html.bootstrap {

    exports org.nasdanika.sdk.runtime.models.html.bootstrap;
    exports org.nasdanika.sdk.runtime.models.html.bootstrap.impl;
    exports org.nasdanika.sdk.runtime.models.html.bootstrap.util;

    exports org.nasdanika.sdk.runtime.models.html.bootstrap.forms;
    exports org.nasdanika.sdk.runtime.models.html.bootstrap.forms.impl;
    exports org.nasdanika.sdk.runtime.models.html.bootstrap.forms.util;

    exports org.nasdanika.sdk.runtime.models.html.bootstrap.components.content;
    exports org.nasdanika.sdk.runtime.models.html.bootstrap.components.content.impl;
    exports org.nasdanika.sdk.runtime.models.html.bootstrap.components.content.util;

    exports org.nasdanika.sdk.runtime.models.html.bootstrap.components.navigation;
    exports org.nasdanika.sdk.runtime.models.html.bootstrap.components.navigation.impl;
    exports org.nasdanika.sdk.runtime.models.html.bootstrap.components.navigation.util;

    exports org.nasdanika.sdk.runtime.models.html.bootstrap.components.overlays;
    exports org.nasdanika.sdk.runtime.models.html.bootstrap.components.overlays.impl;
    exports org.nasdanika.sdk.runtime.models.html.bootstrap.components.overlays.util;

    exports org.nasdanika.sdk.runtime.models.html.bootstrap.generator;
    exports org.nasdanika.sdk.runtime.models.html.bootstrap.builders;

    requires transitive org.nasdanika.sdk.runtime.models.html;

    provides EPackage with
        BootstrapEPackageProvider,
        FormsEPackageProvider,
        ContentEPackageProvider,
        NavigationEPackageProvider,
        OverlaysEPackageProvider;

}
```

Both provide their packages as services, so a resource set configured from the module path loads and saves page models in every format the runtime supports.

3.3. Names that clash

The HTML model's classes have the names of HTML's elements: `Document`, `Element`, `Node`, `Text`, `Image`, `Table`, `Form`. `Document`, `Element`, `Node` and `Text` are also in `org.w3c.dom`, and the Bootstrap model has its own `Container`, `Image`, `Figure`, `Table` and `Button`, more specific than HTML's. The listings of this book

create objects through the factories, so the imports rarely matter.

3.4. What the build does

The Xcore definitions are `html.xcore` in the HTML model, and `bootstrap.xcore`, `forms.xcore`, and `components/content.xcore`, `navigation.xcore` and `overlays.xcore` in the Bootstrap model. Maven generates the Java sources into `src-gen` with the Xtext Maven plugin. The generators, `BootstrapStructure` and `NavigationTree` are hand-written.

Chapter 4. The HTML model

4.1. Nodes and elements

Everything that can be placed into an element is a `Node`, and every node is `Marked`, with markers to the elements of a source model it came from.

Node	What it writes
`Text`	Text, escaped
`Markup`	HTML, verbatim, without escaping: an escape hatch for content the model does not cover, such as the output of a Markdown renderer
`Comment`	An HTML comment
`NodeReference`	Another node, contained elsewhere: the same fragment in several places
`Element`	An element

An `Element` has an `id`, `classes`, a `style` map from properties to values, and `attributes`: a map for the attributes the element's class does not model, `data- and aria-` among them, where a null value is a boolean attribute.

Elements with content implement `Container`, whose `contents` are nodes. The model does not encode the rules of HTML's content model, such as no `div` in a `p`, in its types.

The model has a class per element that pages commonly need, with that element's attributes as features: documents, sections and headings, paragraphs and phrasing, links and images, lists, tables, and forms. `GenericElement` and `GenericVoidElement` write any other tag, custom elements included.

4.2. The document

A `Document` is the `html` element preceded by the doctype, with `lang`, a `head` and a `body`. The `Head` has a `charset`, `utf-8` by default and written first, a `title`, a `base`, `metas`, `links`, `stylesheets`, an `importMap`, and `scripts`. A `Stylesheet` with an `href` is a `link`; with `content`, a `style` element. A `Script` has `src`, `content` and `module`.

Listing 3. A document (`TestGenerator.java`, in the HTML model)

```
Document document = FACTORY.createDocument();
document.setLang("en");

Head head = FACTORY.createHead();
document.setHead(head);
head.setTitle("Tom & Jerry");
Meta viewport = FACTORY.createMeta();
viewport.setName("viewport");
viewport.setContent("width=device-width, initial-scale=1");
head.getMetas().add(viewport);
Stylesheet external = FACTORY.createStylesheet();
external.setHref("styles.css");
head.getStylesheets().add(external);
Stylesheet inline = FACTORY.createStylesheet();
inline.setContent("p > b { color: red; }");
head.getStylesheets().add(inline);
Script script = FACTORY.createScript();
script.setModule(true);
script.setSrc("main.js");
head.getScripts().add(script);

Body body = FACTORY.createBody();
document.setBody(body);
Div div = FACTORY.createDiv();
div.setId("main");
div.getClasses().add("container");
div.getClasses().add("wide");
div.getStyle().put("margin", "0 auto");
body.getContents().add(div);

Heading heading = FACTORY.createHeading();
heading.setLevel(2);
text(heading, "Hello");
div.getContents().add(heading);

// Mixed content stays on one line
Paragraph paragraph = FACTORY.createParagraph();
text(paragraph, "Hello, ");
Strong strong = FACTORY.createStrong();
text(strong, "World");
paragraph.getContents().add(strong);
text(paragraph, "!");
div.getContents().add(paragraph);

UnorderedList list = FACTORY.createUnorderedList();
ListItem item = FACTORY.createListItem();
text(item, "One");
list.getItems().add(item);
div.getContents().add(list);

assertThat(new Generator().generate(document)).isEqualTo("""
        <!DOCTYPE html>
```

```
        <html lang="en">
          <head>
            <meta charset="utf-8">
            <title>Tom &amp;amp; Jerry</title>
            <meta name="viewport" content="width=device-width, initial-scale=1">
            <link rel="stylesheet" href="styles.css">
            <style>p > b { color: red; }</style>
            <script src="main.js" type="module"></script>
          </head>
          <body>
            <div id="main" class="container wide" style="margin: 0 auto;">
              <h2>Hello</h2>
              <p>Hello, <strong>World</strong>!</p>
              <ul>
                <li>One</li>
              </ul>
            </div>
          </body>
        </html>""");
```

The generator indents block elements and keeps mixed content on one line, as the paragraph with the bold "World" shows. `setIndent(null)` writes everything on one line.

4.3. Attributes and escaping

Attributes of an element's class are written when they are set: strings and numbers as `name="value"`, booleans as a bare attribute when true, and not at all when false.

Listing 4. Attributes and escaping (`TestGenerator.java`)

```
Div div = FACTORY.createDiv();
div.getAttributes().put("data-title", "\"quoted\" & <tagged>");
div.getAttributes().put("hidden", null);

Label label = FACTORY.createLabel();
label.setHtmlFor("name");
text(label, "Name");
div.getContents().add(label);

Input input = FACTORY.createInput();
input.setId("name");
input.setType("text");
input.setRequired(true);
input.setDisabled(false);
div.getContents().add(input);

Image image = FACTORY.createImage();
image.setSrc("logo.png");
image.setAlt("Logo");
```

```
image.setWidth(32);
div.getContents().add(image);

Select select = FACTORY.createSelect();
select.setName("color");
Option option = FACTORY.createOption();
option.setValue("r");
option.setSelected(true);
option.setText("Red & Rose");
select.getItems().add(option);
div.getContents().add(select);

Script script = FACTORY.createScript();
script.setContent("document.write('</script>');");
div.getContents().add(script);

assertThat(new Generator().setIndent(null).generate(div)).isEqualTo(
        "<div data-title=\"&quot;quoted&quot; &amp; &lt;tagged&gt;\" hidden>"
        + "<label for=\"name\">Name</label>"
        + "<input id=\"name\" type=\"text\" required>"
        + "<img src=\"logo.png\" alt=\"Logo\" width=\"32\">"
        + "<select name=\"color\"><option value=\"r\" selected>Red &amp; Rose</option></select>"
        + "<script>document.write('<\\/script>');</script>"
        + "</div>");
```

The `data-title` value is escaped for an attribute, the option's text for content, and the script's content so that `</script>` in it does not end the element.

4.4. Import maps

An `ImportMap` in the head maps bare module specifiers, such as `react`, to URLs, globally and in scopes for prefixes of URLs, and is written as a `script` of type `importmap` before the scripts that need it. That is how a page runs ES modules from a CDN with no bundler; the runtime's React model generates applications that way.

4.5. The generator and the structure

`Generator` writes a node, a document or a fragment, to a string or an `Appendable`. It asks an `HtmlStructure` what a model element is in HTML: its tag name, whether it is void, its classes, attributes and children. `HtmlStructure` reads the model reflectively, with two hints on features in the annotation `urn:org.nasdanika:html`: `attribute` for an HTML name that differs from the feature's name, such as `http-equiv`, and `logical` for features that are not attributes, such as a heading's level.

Subclasses of `HtmlStructure` add what a library built on HTML needs. `BootstrapStructure` is one; the next two chapters use it.

4.6. Library elements

`LibraryElement` is the base of the elements of a library built on HTML, Bootstrap's among them. Its classes are written from annotations, looked up on the class and then its supertypes:

- `tag`, the tag name, and `classes`, the classes every element of the class has;
- on a feature, a pattern of classes, `btn-{}`, in which the value replaces `{}`, or, for a boolean, a class that is there when it is true;
- `child`, for a string feature written as a text child.

A library's model is therefore annotated Xcore, with code only for what annotations cannot say.

Chapter 5. Documents, themes and layout

The listings of this chapter and the next build models with a small helper of the tests, `Bootstrap`: its constants are the factories of the five packages and of HTML (`CORE`, `CONTENT`, `NAVIGATION`, `OVERLAYS`, `FORMS`, `HTML`), and `with(object, configurator)` configures a new object where it is created. Each listing ends by comparing what `BootstrapStructure.generate` writes with a file, shown after it.

5.1. The vocabulary

The core package has the words the components share:

Enumeration	Literals
`Variant`	The theme colors: `primary`, `secondary`, `success`, `danger`, `warning`, `info`, `light`, `dark`
`ButtonVariant`	The variants and `link`, and their outline forms, such as `outline-primary`
`Size`	`sm`, `lg`
`Breakpoint`	`xs`, `sm`, `md`, `lg`, `xl`, `xxl`
`ColorMode`	`light`, `dark`: `data-bs-theme` of a document or an element
`Theme`	`bootstrap` and the Bootswatch themes, from `brite` to `zephyr`

Every Bootstrap element extends `BootstrapElement`, a library element, so it also has HTML's `id`, `classes`, `style` and `attributes`; classes set by hand are added to those the element contributes.

5.2. A document with a theme

A `BootstrapDocument` is an HTML document with a `theme` and a `colorMode`. The generator adds the theme's stylesheet, Bootstrap's script, and the initialization of tooltips and popovers:

Listing 5. A themed document (`BootstrapGeneratorTests.java`)

```
var document = with(CORE.createBootstrapDocument(), d -> {
    d.setTheme(Theme.FLATLY);
    d.setColorMode(ColorMode.DARK);
    d.setLang("en");
    d.setHead(with(HTML.createHead(), h -> h.setTitle("Themed")));
    var trigger = with(OVERLAYS.createOverlayTrigger(), t -> {
        t.setPlacement(Placement.TOP);
        t.setOverlay(with(OVERLAYS.createTooltip(), tip -> tip.setTitle("Tip")));
```

```
        t.setTarget(with(CONTENT.createButton(), b -> b.setContent("Hover")));
    });
    d.setBody(with(HTML.createBody(), b -> b.getContents().add(trigger)));
});
Expected.assertMatches("document.html", generate(document));
```

```
<!DOCTYPE html>
<html lang="en" data-bs-theme="dark">
  <head>
    <meta charset="utf-8">
    <title>Themed</title>
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootswatch@5.3.8/dist/flatly/bootstrap.min.css">
  </head>
  <body><span class="d-inline-block" tabindex="0" data-bs-toggle="tooltip" data-bs-title="Tip"
data-bs-placement="top"><button class="btn btn-primary" type="button">
Hover</button></span><script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"></script><scri
pt>document.querySelectorAll('[data-bs-toggle="tooltip"]').forEach(element => new bootstrap
.Tooltip(element));
document.querySelectorAll('[data-bs-toggle="popover"]').forEach(element => new bootstrap.Popover
(element));
</script></body>
</html>
```

The stylesheet and the script come from jsDelivr, at pinned versions of Bootstrap and Bootswatch.

5.3. The grid

`Container`, `Row` and `Col` are Bootstrap's grid; `Stack` is a vertical or horizontal stack with a gap. Sizes, offsets and the number of columns per row are values per breakpoint, which the structure turns into Bootstrap's responsive classes:

Listing 6. A responsive grid (`BootstrapGeneratorTests.java`)

```
var container = with(CORE.createContainer(), c -> c.setFluidUntil(Breakpoint.MD));
container.getRows().add(with(CORE.createRow(), r -> {
    r.getColumnsPerRow().add(with(CORE.createRowColumns(), rc -> {
        rc.setBreakpoint(Breakpoint.XS);
        rc.setCount(1);
    }));
    r.getColumnsPerRow().add(with(CORE.createRowColumns(), rc -> {
        rc.setBreakpoint(Breakpoint.MD);
        rc.setCount(3);
    }));
    r.setGutter(3);
```

```
    r.getColumns().add(with(CORE.createCol(), c -> {
        c.getSizes().add(size(Breakpoint.XS, 12));
        c.getSizes().add(size(Breakpoint.MD, 4, 2));
        c.getContents().add(Bootstrap.text("First"));
    }));
    r.getColumns().add(with(CORE.createCol(), c -> {
        c.getSizes().add(with(CORE.createColumnSize(), s -> {
            s.setBreakpoint(Breakpoint.MD);
            s.setAuto(true);
        }));
        c.getContents().add(with(CORE.createStack(), s -> {
            s.setHorizontal(true);
            s.setGap(2);
            s.getContents().add(Bootstrap.text("Stacked"));
        }));
    }));
    r.getColumns().add(CORE.createCol());
}));
Expected.assertMatches("grid.html", generate(container));
```

```
<div class="container-md">
  <div class="row row-cols-1 row-cols-md-3 g-3">
    <div class="col-12 col-md-4 offset-md-2">First</div>
    <div class="col-md-auto">
      <div class="hstack gap-2">Stacked</div>
    </div>
    <div class="col"></div>
  </div>
</div>
```

A container that is fluid until a breakpoint is `container-md`; a column 12 wide on the smallest screens and 4 wide with an offset of 2 from the medium breakpoint is `col-12 col-md-4 offset-md-2`.

The core package also has `Ratio`, for content with an aspect ratio, `Image`, with its responsive and thumbnail forms, and `Figure`, with its image and caption.

Chapter 6. Components

The components are in four packages: content, navigation and overlays under `components`, and forms.

6.1. Content

Alerts, badges, buttons and their groups and toolbars, cards, list groups, tables, progress bars, spinners, placeholders, accordions and carousels:

Listing 7. Content components (`BootstrapGeneratorTests.java`)

```
var card = with(CONTENT.createCard(), c -> {
    c.setBorder(Variant.PRIMARY);
    c.setHeader(with(CONTENT.createCardHeader(), h -> h.setContent("Header")));
    c.setImage(with(CONTENT.createCardImage(), i -> {
        i.setVariant(CardImageVariant.TOP);
        i.setSrc("card.png");
        i.setAlt("Card");
    }));
    c.getBodies().add(with(CONTENT.createCardBody(), b -> {
        b.setTitle(with(CONTENT.createCardTitle(), t -> t.setContent("Title")));
        b.getTexts().add(with(CONTENT.createCardText(), t -> t.setContent("Text & more")));
        b.getLinks().add(with(CONTENT.createCardLink(), l -> {
            l.setHref("#more");
            l.setContent("More");
        }));
    }));
});
var alert = with(CONTENT.createAlert(), a -> {
    a.setVariant(Variant.WARNING);
    a.setDismissible(true);
    a.setHeading(with(CONTENT.createAlertHeading(), h -> h.setContent("Heads up")));
    a.setContent("Dismiss me");
});
var progress = with(CONTENT.createProgressBar(), p -> {
    p.setNow(60);
    p.setLabel("60%");
    p.setVariant(Variant.SUCCESS);
    p.setStriped(true);
});
var spinner = with(CONTENT.createSpinner(), s -> s.setContent("Loading..."));
var accordion = with(CONTENT.createAccordion(), a -> {
    for (String key: new String[] { "one", "two" }) {
        a.getItems().add(with(CONTENT.createAccordionItem(), i -> {
            i.setEventKey(key);
            i.setHeader(with(CONTENT.createAccordionHeader(), h -> {
                h.setContent("Item " + key);
```

```
            }));
            i.setBody(with(CONTENT.createAccordionBody(), b -> b.setContent("Body " + key)));
        }));
    }
});
var stack = with(CORE.createStack(), s -> s.getContents()
        .addAll(List.of(card, alert, progress, spinner, accordion)));
Expected.assertMatches("content.html", generate(stack));
```

```
<div class="vstack">
  <div class="card border-primary"><div class="card-header">Header</div><img class="card-img-top"
src="card.png" alt="Card"><div class="card-body"><h5 class="card-title">Title</h5><p class="card-
text">Text &amp;amp; more</p><a class="card-link" href="#more">More</a></div></div>
  <div class="alert alert-warning alert-dismissible fade show" role="alert"><h4 class="alert-
heading">Heads up</h4>Dismiss me<button class="btn-close" type="button" data-bs-dismiss="alert"
aria-label="Close"></button></div>
  <div class="progress" role="progressbar" aria-label="60%" aria-valuenow="60" aria-valuemin="0"
aria-valuemax="100">
    <div class="progress-bar bg-success progress-bar-striped" style="width: 60%;">60%</div>
  </div>
  <div class="spinner-border" role="status"><span class="visually-hidden">Loading...</span></div>
  <div id="accordion-contents-4" class="accordion">
    <div class="accordion-item">
      <h2 class="accordion-header"><button class="accordion-button" type="button" data-bs-
toggle="collapse" data-bs-target="#accordion-contents-4-one" aria-expanded="true" aria-
controls="accordion-contents-4-one">Item one</button></h2>
      <div id="accordion-contents-4-one" class="accordion-collapse collapse show" data-bs-
parent="#accordion-contents-4">
        <div class="accordion-body">Body one</div>
      </div>
    </div>
    <div class="accordion-item">
      <h2 class="accordion-header"><button class="accordion-button collapsed" type="button" data-
bs-toggle="collapse" data-bs-target="#accordion-contents-4-two" aria-expanded="false" aria-
controls="accordion-contents-4-two">Item two</button></h2>
      <div id="accordion-contents-4-two" class="accordion-collapse collapse" data-bs-
parent="#accordion-contents-4">
        <div class="accordion-body">Body two</div>
      </div>
    </div>
  </div>
</div>
```

The card's parts are features, not children in a particular order: the header, the image with its position, the bodies with their titles, texts and links. The structure writes them in Bootstrap's order. The accordion's items get ids from their position, and the buttons of their headers point at them.

6.2. Forms

Forms, groups of controls with labels, controls, selects, checks and switches, ranges, floating labels, input groups, and feedback for validation:

Listing 8. A form (BootstrapGeneratorTests.java)

```
var form = with(FORMS.createForm(), f -> {
    f.setNovalidate(true);
    f.getGroups().add(with(FORMS.createFormGroup(), g -> {
        g.setControlId("email");
        g.getClasses().add("mb-3");
        g.setLabel(with(FORMS.createFormLabel(), l -> l.setContent("Email")));
        g.getControls().add(with(FORMS.createFormControl(), c -> {
            c.setType("email");
            c.setRequired(true);
            c.setPlaceholder("name@example.com");
        }));
        g.getFeedback().add(with(FORMS.createFormFeedback(), fb -> {
            fb.setType(FeedbackType.INVALID);
            fb.setContent("Enter an email");
        }));
    }));
    f.getGroups().add(with(FORMS.createFormGroup(), g -> {
        g.getControls().add(with(FORMS.createFormCheck(), c -> {
            c.setType(CheckType.SWITCH);
            c.setInputId("dark");
            c.setLabel("Dark mode");
            c.setChecked(true);
        }));
        g.getControls().add(with(FORMS.createFloatingLabel(), fl -> {
            fl.setControlId("city");
            fl.setLabel("City");
            fl.setControl(with(FORMS.createFormControl(), c -> c.setPlaceholder("City")));
        }));
        g.getControls().add(with(FORMS.createFormSelect(), s -> {
            s.setName("color");
            s.getOptions().add(with(HTML.createOption(), o -> {
                o.setValue("red");
                o.setText("Red");
            }));
        }));
    }));
});
Expected.assertMatches("forms.html", generate(form));
```

```
<form novalidate>
  <div class="mb-3"><label class="form-label" for="email">Email</label><input id="email"
class="form-control" type="email" required placeholder="name@example.com"><div class="invalid-
feedback">Enter an email</div></div>
  <div>
```

```
    <div class="form-check form-switch"><input id="dark" class="form-check-input" type="checkbox"
role="switch" checked><label class="form-check-label" for="dark">Dark mode</label></div>
    <div class="form-floating"><input id="city" class="form-control" type="text"
placeholder="City"><label for="city">City</label></div>
    <select class="form-select" name="color">
      <option value="red">Red</option>
    </select>
  </div>
</form>
```

A group's `controlId` becomes the control's `id` and the label's `for`, so they cannot disagree.

6.3. Navigation

Navs and their links and dropdowns, navbars with a brand, a toggle and a collapse, tabs, breadcrumbs, pagination, dropdowns and split buttons:

Listing 9. Navigation components (`BootstrapGeneratorTests.java`)

```
var navbar = with(NAVIGATION.createNavbar(), n -> {
    n.setBg(Variant.DARK);
    n.setTheme(ColorMode.DARK);
    n.setExpand(Breakpoint.LG);
    n.getContent().add(with(CORE.createContainer(), c -> {
        c.setFluid(true);
        c.getContents().add(with(NAVIGATION.createNavbarBrand(), b -> {
            b.setHref("#");
            b.setContent("Nasdanika");
        }));
        c.getContents().add(with(NAVIGATION.createNavbarToggle(), t -> {
            t.setControls("main-nav");
        }));
        c.getContents().add(with(NAVIGATION.createNavbarCollapse(), collapse -> {
            collapse.setId("main-nav");
            collapse.getContent().add(with(NAVIGATION.createNav(), nav -> {
                nav.getEntries().add(with(NAVIGATION.createNavLink(), l -> {
                    l.setHref("#home");
                    l.setActive(true);
                    l.setContent("Home");
                }));
                nav.getEntries().add(with(NAVIGATION.createNavDropdown(), d -> {
                    d.setTitle("More");
                    d.getEntries().add(with(NAVIGATION.createDropdownItem(), i -> {
                        i.setHref("#a");
                        i.setContent("A");
                    }));
                    d.getEntries().add(NAVIGATION.createDropdownDivider());
                }));
            }));
```

```
        }));
    }));
});
var tabs = with(NAVIGATION.createTabs(), t -> {
    t.setId("models");
    t.getTabs().add(with(NAVIGATION.createTab(), tab -> {
        tab.setEventKey("html");
        tab.setTitle("HTML");
        tab.setContent("HTML pane");
    }));
    t.getTabs().add(with(NAVIGATION.createTab(), tab -> {
        tab.setEventKey("react");
        tab.setTitle("React");
        tab.setContent("React pane");
    }));
});
var pagination = with(NAVIGATION.createPagination(), p -> {
    p.getEntries().add(NAVIGATION.createPaginationPrev());
    p.getEntries().add(with(NAVIGATION.createPaginationItem(), i -> {
        i.setContent("1");
        i.setActive(true);
    }));
    p.getEntries().add(NAVIGATION.createPaginationNext());
});
var stack = with(CORE.createStack(), s -> s.getContents()
        .addAll(List.of(navbar, tabs, pagination)));
Expected.assertMatches("navigation.html", generate(stack));
```

```
<div class="vstack">
  <nav class="navbar bg-dark navbar-expand-lg" data-bs-theme="dark">
    <div class="container-fluid"><a class="navbar-brand" href="#">Nasdanika</a><button
class="navbar-toggler" type="button" data-bs-toggle="collapse" aria-expanded="false" aria-
label="Toggle navigation" data-bs-target="#main-nav" aria-controls="main-nav"><span
class="navbar-toggler-icon"></span></button><div id="main-nav" class="collapse navbar-collapse">
        <ul class="navbar-nav">
          <li class="nav-item"><a class="nav-link active" href="#home" aria-current="page">
Home</a></li>
          <li class="nav-item dropdown"><a class="nav-link dropdown-toggle" href="#"
role="button" data-bs-toggle="dropdown" aria-expanded="false">More</a><ul class="dropdown-menu">
              <li><a class="dropdown-item" href="#a">A</a></li>
              <li>
                <hr class="dropdown-divider">
              </li>
            </ul></li>
        </ul>
      </div></div>
  </nav>
  <div id="models">
    <ul class="nav nav-tabs" role="tablist">
      <li class="nav-item" role="presentation"><button id="models-html-tab" class="nav-link
active" data-bs-toggle="tab" data-bs-target="#models-html-pane" type="button" role="tab" aria-
controls="models-html-pane" aria-selected="true">HTML</button></li>
      <li class="nav-item" role="presentation"><button id="models-react-tab" class="nav-link"
```

```
data-bs-toggle="tab" data-bs-target="#models-react-pane" type="button" role="tab" aria-
controls="models-react-pane" aria-selected="false">React</button></li>
    </ul>
    <div class="tab-content">
      <div id="models-html-pane" class="tab-pane fade show active" role="tabpanel" aria-
labelledby="models-html-tab" tabindex="0">HTML pane</div>
      <div id="models-react-pane" class="tab-pane fade" role="tabpanel" aria-labelledby="models-
react-tab" tabindex="0">React pane</div>
    </div>
  </div>
  <nav aria-label="Pagination">
    <ul class="pagination">
      <li class="page-item"><a class="page-link" href="#" aria-label="Previous"><span aria-
hidden="true">&laquo;</span></a></li>
      <li class="page-item active"><a class="page-link" href="#" aria-current="page">1</a></li>
      <li class="page-item"><a class="page-link" href="#" aria-label="Next"><span aria-
hidden="true">&raquo;</span></a></li>
    </ul>
  </nav>
</div>
```

Tabs are a list of tabs, each with a title and a pane; the structure writes the buttons and the panes, and connects them by ids.

6.4. Overlays

Modals, offcanvases, toasts, and tooltips and popovers on the elements they decorate:

Listing 10. Overlays (BootstrapGeneratorTests.java)

```
var modal = with(OVERLAYS.createModal(), m -> {
    m.setCentered(true);
    m.setHeader(with(OVERLAYS.createModalHeader(), h -> {
        h.setCloseButton(true);
        h.setTitle(with(OVERLAYS.createModalTitle(), t -> t.setContent("Title")));
    }));
    m.setBody(with(OVERLAYS.createModalBody(), b -> b.setContent("Body")));
});
var toggle = with(OVERLAYS.createToggle(), t -> {
    t.setTarget(modal);
    t.setContent("Open");
});
var popover = with(OVERLAYS.createOverlayTrigger(), t -> {
    t.setOverlay(with(OVERLAYS.createPopover(), p -> {
        p.setTitle("Popover");
        p.setContent("Content");
    }));
    t.setTarget(element(HTML.createSpan(), "Click"));
});
```

```
var stack = with(CORE.createStack(), s -> s.getContents()
        .addAll(List.of(toggle, modal, popover)));
Expected.assertMatches("overlays.html", generate(stack));
```

```
<div class="vstack"><button class="btn btn-primary" type="button" data-bs-toggle="modal" data-bs-
target="#modal-contents-1" aria-controls="modal-contents-1">Open</button><div id="modal-contents-
1" class="modal fade" tabindex="-1" aria-hidden="true" aria-labelledby="modaltitle-contents-1-
header-title">
    <div class="modal-dialog modal-dialog-centered">
      <div class="modal-content">
        <div class="modal-header"><h1 id="modaltitle-contents-1-header-title" class="modal-title
fs-5">Title</h1><button class="btn-close" type="button" data-bs-dismiss="modal" aria-
label="Close"></button></div>
        <div class="modal-body">Body</div>
      </div>
    </div>
  </div><span class="d-inline-block" tabindex="0" data-bs-toggle="popover" data-bs-
title="Popover" data-bs-content="Content"><span>Click</span></span></div>
```

A `Toggle` refers to the modal it opens, and the structure writes the button's `data-bs-toggle`, `data-bs-target` and `aria-controls` from the reference. An `OverlayTrigger` wraps its target in a `span` that carries the tooltip or popover, which works for disabled buttons too.

6.5. The demo site

The module's `BootstrapDemoTests` generates a static site from the model: a page per package, showing every component, and a gallery of the same showcase in every Bootstrap and Bootswatch theme. It is written to `target/html-bootstrap/site`; browse it with the JDK's `jwebserver -d target/html-bootstrap/site`.

Chapter 7. Navigation trees

Documentation sites of models need navigation that works for people, crawlers and agents alike. `NavigationTree`, in the Bootstrap module's `builders` package, builds a side panel as nested lists of `details` and `summary` elements, which work without JavaScript, so crawlers and agents read the tree as plain links.

- Branches on the path to the current item are open, and the current item is marked with `aria-current="page"`.
- A click on a label follows its link; a click on the chevron opens or closes the branch.
- Items have optional links and icons: CSS classes, such as `bi bi-filetype-java` from Bootstrap Icons, with a different icon for an open branch if wanted.
- The tree's styles use Bootstrap's CSS variables, so it follows the theme and the color mode.

Listing 11. A tree, and a local tree (`NavigationTreeDemoTests.java`)

```
NavigationTree tree = new NavigationTree("Family");
NavigationTree.Item family = tree.add("Family");
NavigationTree.Item members = family.add("Members").open();
members.add("Alain");
members.add("Bob");
family.add("Pets").add("Rex");
String html = BootstrapStructure.generate(tree.build());
assertTrue(html.contains("<details open"), html);
assertTrue(html.contains("<span class=\"nav-tree-label\">Rex</span>"), html);
assertTrue(!html.contains("<i "), html);

tree.local(true);
html = BootstrapStructure.generate(tree.build());
assertTrue(html.contains("nav-tree-collapsed"), html);
assertTrue(!html.contains("Rex"), html);
```

The tree needs `NavigationTree.stylesheet()` in the head, and `iconsStylesheet()` for Bootstrap Icons.

7.1. Large models

A page of a model with thousands of elements cannot carry the whole tree. A local tree, `local(true)`, writes the branches off the path to the current item as links without their children, so each page stays small and the tree is still complete when followed.

7.2. The demo

`NavigationTreeDemoTests.testTree` builds a site of the module's own sources: a page per file and directory, each with the tree, the current file open, and the file's content. It is written to `target/html-bootstrap/site/tree`, next to the demo site of Chapter 6, and shows the tree on a few hundred pages.

Part III: Looking Ahead

Documents, documentation sites for models, and pages that work offline.

Chapter 8. Looking ahead

Everything in this chapter is **PLANNED**.

8.1. Documents

A document model as a pivot: structure in the model, text in a named format. Markdown comes in through a ported model of its syntax, and documents go out as AsciiDoc, and as Bootstrap pages through the models of this book.

8.2. Documentation sites for models

A page per element of any model built on the runtime's core model: its documentation, its neighbors, links to where it came from, and a diagram of its neighborhood, generated as SVG, with every shape a link. The navigation trees of Chapter 7 are their side panels.

8.3. Pages that work offline

The stylesheets and scripts of Bootstrap, Bootswatch and Bootstrap Icons copied into the site, with the pages pointing at the copies, for offline use and closed networks.

8.4. The polymorphic UI

Views chosen by the class of the element they show and the format asked for, assembled into pages and applications, static with this book's models and interactive with React's.

Appendix A: Reference

A.1. Packages

Package	Namespace URI
org.nasdanika.sdk.runtime.models.html	runtime.sdk.nasdanika.org/models/html
org.nasdanika.sdk.runtime.models.html.bootstrap	runtime.sdk.nasdanika.org/models/html/bootstrap
⋯bootstrap.forms	runtime.sdk.nasdanika.org/models/html/bootstrap/forms
⋯bootstrap.components.content	runtime.sdk.nasdanika.org/models/html/bootstrap/components/content
⋯bootstrap.components.navigation	runtime.sdk.nasdanika.org/models/html/bootstrap/components/navigation
⋯bootstrap.components.overlays	runtime.sdk.nasdanika.org/models/html/bootstrap/components/overlays

A.2. Bootstrap classes

Package	Classes
core	BootstrapDocument, Container, Row, Col, ColumnSize, RowColumns, Stack, Ratio, Image, Figure, FigureImage, FigureCaption
forms	Form, FormGroup, FormLabel, FormControl, FormTextArea, FormSelect, FormCheck, FormRange, FormText, FormFeedback, FloatingLabel, InputGroup, InputGroupText
content	Alert, AlertHeading, AlertLink, Badge, Button, ButtonGroup, ButtonToolbar, CloseButton, Card and its parts, CardGroup, ListGroup, ListGroupItem, Table, ProgressBar, Spinner, Placeholder, PlaceholderGroup, PlaceholderButton, Accordion and its parts, Carousel, CarouselItem, CarouselCaption
navigation	Nav, NavItem, NavLink, NavDropdown, Navbar, NavbarBrand, NavbarToggle, NavbarCollapse, NavbarText, Tabs, Tab, Breadcrumb, BreadcrumbItem, Pagination and its items, Dropdown, DropdownToggle, DropdownMenu, DropdownItem, DropdownHeader, DropdownDivider, DropdownItemText, DropdownButton, SplitButton
overlays	Toggle, Modal and its parts, Offcanvas and its parts, Toast, ToastHeader, ToastBody, ToastContainer, Tooltip, Popover, OverlayTrigger

A.3. Java classes

Class	What it is for
`html.generator.Generator`	Writes a node as HTML. `setIndent(null)` writes on one line
`html.generator.HtmlStructure`	Maps the HTML model to tags, attributes and children
`bootstrap.generator.BootstrapStructure`	Maps the Bootstrap model, expanding components into HTML. `generate(node)` writes a node; `stylesheet(theme, …)` and `javaScript(…)` give the URLs of the CDN
`bootstrap.builders.NavigationTree`	Builds a navigation tree of `details` and `summary`

Appendix B: Building this book

This book is AsciiDoc in `models/html/bootstrap/src/docs/asciidoc`, built with the Bootstrap model after its tests have passed:

```
mvn -pl models/html/bootstrap -am verify                          # the model, its tests and this
book
mvn -pl models/html/bootstrap verify -Dzephyr.editor-notes=false  # without editor's notes
mvn -pl models/html/bootstrap verify -Dasciidoctor.skip=true      # without the book
```

The PDF and the single-page HTML are written to `target/book` and attached to the Bootstrap model's artifact with the classifier `book`; the test sources are attached with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/models/html/bootstrap/tests`, of the HTML model's tests in `models/html/model/src/test/java/org/nasdanika/sdk/runtime/models/html/tests`, and of the module declarations. A listing line holds at most 96 characters, and a warning fails the build.

The HTML after each Bootstrap listing is a file in `src/test/resources/expected`, which the test compares with what the generator writes. The test also writes what it generated to `target/expected-actual`; to change an expectation, review the difference between the two files, and copy the actual one over the expected one.

Appendix C: Glossary

Breakpoint

A minimum width of the screen at which a responsive class applies: `sm`, `md`, `lg`, `xl`, `xxl`.

Bootswatch

A collection of free themes for Bootstrap, of which this book's look, Zephyr, is one.

Import map

A JSON map in a page from bare module specifiers to URLs, which lets a browser load ES modules by package name.

Library element

An element of the HTML model whose class is written from annotations: a component of a library built on HTML.

Structure

The object that tells the generator what a model element is in HTML: `HtmlStructure`, or a subclass for a library.

Void element

An HTML element without content and without an end tag, such as `img` or `input`.